Lab Report:1

OOP in Python (Constructors, Inheritance, Modular Programming)

Objectives

- To understand **Object-Oriented Programming (OOP)** in Python.
- To implement **default (non-parameterized) constructor** and **parameterized constructor**.
- To implement **inheritance** (single + multiple).
- To understand **modular programming** using separate Python files (modules).

Theory

A. OOP in Python

OOP is a programming paradigm that organizes code using **classes and objects**. It helps in code reusability, clarity, and easy maintenance.OOP is a programming approach where we model real-world entities using:

- Class: Blueprint/template (defines properties + behaviors)
- Object: Instance of a class (real usable entity)
- Attributes: Variables inside a class (data)
- Methods: Functions inside a class (behavior)

B. Constructor in Python

A **constructor** is a special method used to initialize an object automatically when an object is created.

1) Default Constructor (Non-parameterized)

- Defined as `__init__(self)`
- Runs automatically when an object is created.
- Used for default initialization.

2) Parameterized Constructor

- Defined as `__init__(self, parameters...)`
- Used to set custom values at object creation time.

C. Inheritance

Inheritance allows a new class (**child/derived class**) to reuse properties and methods of an existing class (**parent/base class**).

- Improves **code reuse**
- Supports **method overriding** (child can redefine parent method)

Types used here:

- **Single Inheritance**: one parent → one child
- **Multiple Inheritance**: child inherits from more than one parent

D. Modular Programming

Modular programming means breaking a program into multiple files (modules) to make code:

- easier to manage
- reusable
- cleaner and organized

Example: one file contains functions (`modular.py`), another file uses them (`main.py`).

Tasks:

 # Class and Object Creation

```
class myclass:
    pass
m1 = myclass()
```

 # Class with Attributes and Method

```
class Myclass:
    name = "avi"
    age = 25
    def func(self):
        print("hello world")
m1 = Myclass()
print(m1.name, m1.age)
m1.func()
```

Output

```
avi 25
hello world
```

 # Default Constructor

```
class Myclass:
    def __init__(self):
        print("hellow world")
m1 = Myclass()
```

Output

```
hellow world
```

 # Parameterized Constructor

```
class Myclass:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
    def display(self):
        print(self.name)
        print(self.age)

m1 = Myclass("aviral", 20)
m1.display()

m2 = Myclass("ram", 22)
m2.display()
```

Output

```
aviral
20
ram
22
```

 # Single Inheritance and Method Overriding

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} make sound")


class Dog(Animal):
    def speak(self):
        print(f"{self.name} barks")
```

```
a = Animal("generic animal")
b = Dog("buddy")
a.speak()
b.speak()
```

Output

```
generic animal make sound
buddy barks
```

 # Multiple Inheritance

```
class living_being:
    def __init__(self, category):
        self.category = category
    def divison(self):
        print(f"{self.category} type")


class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} make sound")


class Dog(living_being, Animal):
    def __init__(self, name, category):
        living_being.__init__(self, category)
        Animal.__init__(self, name)
    def speak(self):
        print(f"{self.name} barks")
```

```
c = living_being("biological")
a = Animal("generic animal")
b = Dog("buddy", "mammal")
c.divison()
a.speak()
b.divison()
b.speak()
```

Output

```
biological type
generic animal make sound
mammal type
buddy barks
```

 # Modular Programming

modular.py

```
def add(a, b):
    return a + b
```

module

main.py

```
import modular as m
result = m.add(2, 3)
print(result)
```

Output

```
5
```

Question)

create object, calculate result and display results using the concept of oop , modular programming

main.py

```
# create object, calculate result, display
from student import student
from operation import calculate_total, calculate_average

# Create student object
s1 = student("Abiral", 101, {"math": 90, "english": 85, "science": 88})

# Display student info
s1.display()

# Calculate and display total and average
total = calculate_total(s1.marks)
average = calculate_average(s1.marks)

print(f"\nTotal Marks: {total}")
print(f"Average Marks: {average}")
```

operation.py

```
# total average
#total and average
import student as s
def calculate_total(marks):
    return sum(marks.values())

def calculate_average(marks):
    total = calculate_total(marks)
    return total / len(marks) if marks else 0
```

student.py

```
# student class
#  attribute name, roll,marks( make dictionary )
# method-->display
class student:
    def __init__(self,name,roll_no,marks):
        self.name=name
        self.roll_no=roll_no
        self.marks=marks  #must be in dictionary{"math":90}
    def display(self):
        print(f"Name: {self.name}")
        print(f"Roll No: {self.roll_no}")
        for key, value in self.marks.items():
            print(f"{key}: {value}")
```

Discussion

In this lab, the fundamental concepts of **Object-Oriented Programming in Python** were studied and implemented successfully. The programs demonstrated how classes and objects are created and how constructors are used to initialize object data. The difference between **default** and **parameterized constructors** was clearly observed through object creation and output results.

The implementation of **inheritance** showed how code reusability and method overriding can be achieved in Python. Single and multiple inheritance helped in understanding how child classes can extend or modify the behavior of parent classes. The use of **modular programming** illustrated how programs can be divided into separate files for better organization and reuse. Overall, the lab enhanced practical understanding of OOP concepts and their importance in developing structured and maintainable Python applications.

Conclusion

This lab helped understand the fundamentals of Object-Oriented Programming in Python, including constructors, inheritance, and modular programming, which are essential for building structured and reusable software.